

15. Transaction Management

Transaction management in JDBC ensures that a set of operations on a database is executed as a single unit. If any operation in the transaction fails, all operations in the transaction can be rolled back to maintain database integrity. JDBC provides control over transactions through the `java.sql.Connection` interface.

If your JDBC Connection is in auto-commit mode, which it is by default, then every SQL statement is committed to the database upon its completion. That may be fine for simple applications, but there are three reasons why you may want to turn off the auto-commit and manage your own transactions –

- To increase performance.
- To maintain the integrity of business processes.
- To use distributed transactions.

Transactions enable you to control if, and when, changes are applied to the database. It treats a single SQL statement or a group of SQL statements as one logical unit, and if any statement fails, the whole transaction fails.

To enable manual- transaction support instead of the auto-commit mode that the JDBC driver uses by default, use the Connection object's `setAutoCommit()` method. If you pass a boolean `false` to `setAutoCommit( )`, you turn off auto-commit. You can pass a boolean `true` to turn it back on again.

For example, if you have a Connection object named `conn`, code the following to turn off auto-commit –

```
connection.setAutoCommit(false);
```

Key Concepts

1. Transaction:
 - A transaction is a sequence of database operations treated as a single unit of work.
 - Properties of a transaction are defined by the ACID principles:
 - Atomicity: All operations must complete successfully; otherwise, the transaction is aborted.
 - Consistency: The database must transition from one valid state to another.
 - Isolation: Transactions must not interfere with each other.
 - Durability: Changes made by a committed transaction are permanent.
2. Auto-commit Mode:
 - By default, JDBC connections are in auto-commit mode, meaning each SQL statement is treated as a transaction and is automatically committed.
 - This mode can be turned off for managing transactions manually.
3. Manual Transaction Management:

- Transactions are explicitly started, committed, or rolled back by the application.

15.1 Commit & Rollback

Once you are done with your changes and you want to commit the changes then call commit() method on connection object as follows –

```
connection.commit();
```

Otherwise, to roll back updates to the database made using the Connection named conn, use the following code –

```
connection.rollback();
```

Creating Table

```
CREATE TABLE Employees (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    first VARCHAR(50) NOT NULL,  
    last VARCHAR(50) NOT NULL,  
    age INT NOT NULL  
);
```

Using Commit Statement to Persist Inserted Rows in Table Example

In this example, we've six static strings containing a database connection url, username, password, SELECT Query and two INSERT queries. Insert Queries are used to insert two records in the Employees table and Select query will be used to get all the employees. Now using DriverManager.getConnection() method, we've prepared a database connection. Using setAutoCommit(false), we've set the auto commit as false which is by default true. Once connection is prepared, we've created a Statement object using connection.createStatement() method. While creating Statement Object, we've used ResultSet types as TYPE_SCROLL_INSENSITIVE and CONCUR_READ_ONLY. Then using statement.executeUpdate(), the INSERT Queries are executed. Now using commit() method, the changes are persisted to the database.

We've defined a method printResultSet() which takes the resultset as argument, iterates it and print all the records of the resultset. Once all changes are committed, printResultSet() method is called to print all the records to verify the inserted records being available or not.

```
package com.lokesh;  
  
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.ResultSet;  
import java.sql.SQLException;  
import java.sql.Statement;  
  
public class JDBCCommitTransactionExample {  
    static final String DB_URL = "jdbc:mysql://localhost/organization";  
    static final String USER = "root";
```

```

static final String PASS = "root";
static final String QUERY = "SELECT id, first, last, age FROM Employees";
static final String INSERT_QUERY = "INSERT INTO Employees (first, last, age)
                                     values('Rita', 'Tez', 20)";
static final String INSERT_QUERY_2 = "INSERT INTO Employees (first, last, age)
                                     values('Sita', 'Singh', 20)";

public static void printResultSet(ResultSet rs) throws SQLException {
    // Ensure we start with first row
    rs.beforeFirst();
    while (rs.next()) {
        // Display values
        System.out.print("ID: " + rs.getInt("id"));
        System.out.print(", Age: " + rs.getInt("age"));
        System.out.print(", First: " + rs.getString("first"));
        System.out.println(", Last: " + rs.getString("last"));
    }
    System.out.println();
}

public static void main(String[] args) {
    Connection conn = null;
    Statement stmt = null;
    try {
        // Open a connection
        System.out.println("Connecting to database...");
        conn = DriverManager.getConnection(DB_URL, USER, PASS);

        // Set auto commit as false.
        conn.setAutoCommit(false);

        // Execute a query to create statement with
        // required arguments for RS example.
        System.out.println("Creating statement...");
        stmt = conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
                                    ResultSet.CONCUR_UPDATABLE);

        // INSERT a row into Employees table
        System.out.println("Inserting one row....");
        stmt.executeUpdate(INSERT_QUERY);

        // INSERT one more row into Employees table
        System.out.println("Inserting another row....");
        stmt.executeUpdate(INSERT_QUERY_2);

        // Commit data here.
        System.out.println("Committing data here....");
        conn.commit();

        // Now list all the available records.
        ResultSet rs = stmt.executeQuery(QUERY);
        System.out.println("List result set for reference....");
        printResultSet(rs);

        // Clean-up environment
        rs.close();
        stmt.close();
        conn.close();
    } catch (SQLException se) {
        se.printStackTrace();
        // If there is an error then rollback the changes.
        System.out.println("Rolling back data here....");
        try {
            if (conn != null)
                conn.rollback();
        }
    }
}

```

```

        } catch (SQLException se2) {
            se2.printStackTrace();
        }
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        // finally block used to close resources
        try {
            if (stmt != null)
                stmt.close();
        } catch (SQLException se2) {
            se2.printStackTrace();
        }
        try {
            if (conn != null)
                conn.close();
        } catch (SQLException se) {
            se.printStackTrace();
        }
    }
}
}

```

Output:

```

Connecting to database...
Creating statement...
Inserting one row....
Inserting another row....
Committing data here....
List result set for reference....
ID: 1, Age: 20, First: Rita, Last: Tez
ID: 2, Age: 20, First: Sita, Last: Singh

```

Using Rollback Statement to Revert Inserted Rows in Table Example

In this example, we've six static strings containing a database connection url, username, password, SELECT Query and two INSERT queries. Insert Queries are used to insert two records in the Employees table and Select query will be used to get all the employees. One of the insert query is invalid as Employee1 table does not exist. Now using DriverManager.getConnection() method, we've prepared a database connection. Using setAutoCommit(false), we've set the auto commit as false which is by default true. Once connection is prepared, we've created a Statement object using connection.createStatement() method. While creating Statement Object, we've used ResultSet types as TYPE_SCROLL_INSENSITIVE and CONCUR_READ_ONLY. Then using statement.executeUpdate(), the INSERT Queries are executed.

As second INSERT query is invalid, SQLException will be raised. We've handled the exception and using rollback() method, the changes are reverted.

We've defined a method printResultSet() which takes the resultset as argument, iterates it and print all the records of the resultset. Once all changes are committed, printResultSet() method

is called to print all the records to verify the inserted records being reverted or not.

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class JDBCRollbackTransactionExample {
    static final String MYSQL_URL = "jdbc:mysql://localhost/organization";
    static final String USER_NAME = "root";
    static final String PASSWORD = "root";
    static final String INSERT_QUERY1 = "INSERT INTO Employees (first, last, age)
                                         values('Jeevan', 'Rao', 35)";
    static final String INSERT_QUERY2 = "INSERT INTO Employees11 (first, last, age)
                                         values('Aditya', 'Chaube', 40)";
    static final String SELECT_QUERY = "SELECT id, age, first, last FROM Employees";

    public static void printResultSet(ResultSet rs) throws SQLException {
        // Ensure we start with first row
        rs.beforeFirst();
        while (rs.next()) {
            // Display values
            System.out.print("ID: " + rs.getInt("id"));
            System.out.print(", Age: " + rs.getInt("age"));
            System.out.print(", First: " + rs.getString("first"));
            System.out.println(", Last: " + rs.getString("last"));
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Connection conn = null;
        Statement stmt = null;
        ResultSet rs = null;
        try {
            // Open database connection
            conn = DriverManager.getConnection(MYSQL_URL, USER_NAME, PASSWORD);
            System.out.println(" Connection established with organization
                                database.");

            conn.setAutoCommit(false);
            stmt = conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
                                       ResultSet.CONCUR_UPDATABLE);

            System.out.println("Before inserting a row into Employees table");
            stmt.executeUpdate(INSERT_QUERY1);
            System.out.println("Inserted a row into Employees table");
            stmt.executeUpdate(INSERT_QUERY2);

            System.out.println("Before committing 2 inserts.");
            conn.commit();
            System.out.println(" After committing 2 inserts.");

            rs = stmt.executeQuery(SELECT_QUERY);
            System.out.println("List ALL records of Employees table..");

            printResultSet(rs);

            // Clean-up resources
            rs.close();
            stmt.close();
            conn.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

```

    } catch (SQLException se) {
        se.printStackTrace();
        System.out.println("If error has occurred, discard changes");
        try {
            if (conn != null)
                conn.rollback();
            // Checking if records are in 'before-insert' state.
            rs = stmt.executeQuery(SELECT_QUERY);
            System.out.println("List ALL records of Employees table..");
            printResultSet(rs);
            // Clean-up resources
            rs.close();
            stmt.close();
            conn.close();
        } catch (SQLException se2) {
            se2.printStackTrace();
            System.out.println("If error has occurred, discard changes");
            try {
                if (conn != null)
                    conn.rollback();

                // Checking if records are in 'before-insert' state.
                rs = stmt.executeQuery(SELECT_QUERY);
                System.out.println("List ALL records of Employees
                                table..");

                printResultSet(rs);
            } catch (SQLException se21) {
                se21.printStackTrace();
            } catch (Exception e) {
                e.printStackTrace();
            } finally {
                try {
                    if (rs != null) {
                        rs.close();
                    }
                } catch (SQLException se31) {
                    se31.printStackTrace();
                }
                try {
                    if (stmt != null)
                        stmt.close();
                } catch (SQLException se32) {
                    se32.printStackTrace();
                }
                try {
                    if (conn != null)
                        conn.close();
                } catch (SQLException se33) {
                    se33.printStackTrace();
                }
            }
        }
    }
}
}
}

```

Output:

```

Connection established with organization database.
Before inserting a row into Employees table
Inserted a row into Employees table
java.sql.SQLException: Table 'organization.employees11' doesn't exist
    at com.mysql.cj.jdbc.exceptions.SQLException.createSQLException(SQLException.java:121)
    at
com.mysql.cj.jdbc.exceptions.SQLExceptionsMapping.translateException(SQLExceptionsMapping.j
ava:122)

```

```

at com.mysql.cj.jdbc.StatementImpl.executeUpdateInternal(StatementImpl.java:1334)
at com.mysql.cj.jdbc.StatementImpl.executeLargeUpdate(StatementImpl.java:2084)
at com.mysql.cj.jdbc.StatementImpl.executeUpdate(StatementImpl.java:1245)
at
com.lokesh.JDBCRollbackTransactionExample.main(JDBCRollbackTransactionExample.java:45)
If error has occurred, discard changes
List ALL records of Employees table..
ID: 1, Age: 20, First: Rita, Last: Tez
ID: 2, Age: 20, First: Sita, Last: Singh

```

15.2 Using Savepoints

The new JDBC 3.0 Savepoint interface gives you the additional transactional control. Most modern DBMS, support savepoints within their environments such as Oracle's PL/SQL.

When you set a savepoint you define a logical rollback point within a transaction. If an error occurs past a savepoint, you can use the rollback method to undo either all the changes or only the changes made after the savepoint.

The Connection object has two new methods that help you manage savepoints –

- `setSavepoint(String savepointName)` – Defines a new savepoint. It also returns a Savepoint object.
- `releaseSavepoint(Savepoint savepointName)` – Deletes a savepoint. Notice that it requires a Savepoint object as a parameter. This object is usually a savepoint generated by the `setSavepoint()` method.

There is one `rollback (String savepointName)` method, which rolls back work to the specified savepoint.

Using SavePoint to rollback to Previous Stable Change Example

In this example, we've six static strings containing a database connection url, username, password, SELECT Query and two DELETE queries. Delete Queries are used to delete two records in the Employees table with id 8 and 9. Now using `DriverManager.getConnection()` method, we've prepared a database connection. Using `setAutoCommit(false)`, we've set the auto commit as false which is by default true. Once connection is prepared, we've created a Statement object using `connection.createStatement()` method. While creating Statement Object, we've used ResultSet types as `TYPE_SCROLL_INSENSITIVE` and `CONCUR_READ_ONLY`.

Using `setSavepoint()` method, we've set a save point and then record with id 8 is deleted. As next step, changes are reverted using `rollback to earlier savepoint`. This will revert the delete row change. Then we've deleted a record with id 9 and records are printed using `printResultSet()` method which prints all the records of a resultset.

```

package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Savepoint;
import java.sql.Statement;

public class SavepointExample1 {
    static final String DB_URL = "jdbc:mysql://localhost/organization";
    static final String USER = "root";
    static final String PASS = "root";
    static final String QUERY = "SELECT id, first, last, age FROM Employees";
    static final String DELETE_QUERY = "DELETE FROM Employees WHERE ID = 8";
    static final String DELETE_QUERY_1 = "DELETE FROM Employees WHERE ID = 9";

    public static void printResultSet(ResultSet rs) throws SQLException {
        // Ensure we start with the first row
        rs.beforeFirst();
        while (rs.next()) {
            // Display values
            System.out.print("ID: " + rs.getInt("id"));
            System.out.print(", Age: " + rs.getInt("age"));
            System.out.print(", First: " + rs.getString("first"));
            System.out.println(", Last: " + rs.getString("last"));
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Connection conn = null;
        Statement stmt = null;
        ResultSet rs = null;

        try {
            // Open connection
            conn = DriverManager.getConnection(DB_URL, USER, PASS);
            conn.setAutoCommit(false); // Set auto-commit to false

            stmt = conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
                                         ResultSet.CONCUR_UPDATABLE);

            // Fetch and display initial data
            rs = stmt.executeQuery(QUERY);
            System.out.println("List result set for reference....");
            printResultSet(rs);

            // Delete row with ID = 8 (set a savepoint first)
            Savepoint savepoint1 = conn.setSavepoint("ROWS_DELETED_1");
            System.out.println("Deleting row....");
            stmt.executeUpdate(DELETE_QUERY);

            // Roll back to savepoint1
            conn.rollback(savepoint1);
            System.out.println("Rolled back to savepoint1");

            // Delete row with ID = 9 (set another savepoint)
            Savepoint savepoint2 = conn.setSavepoint("ROWS_DELETED_2");
            System.out.println("Deleting row....");
            stmt.executeUpdate(DELETE_QUERY_1);

            // Fetch and display updated data
            rs = stmt.executeQuery(QUERY);
            System.out.println("List result set for reference....");
            printResultSet(rs);
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            if (rs != null) rs.close();
            if (stmt != null) stmt.close();
            if (conn != null) conn.close();
        }
    }
}

```

```

        // Commit the changes (if needed)
        conn.commit();

    } catch (SQLException e) {
        // Handle exceptions
        e.printStackTrace();
        if (conn != null) {
            try {
                System.out.println("Rolling back transaction due to an error...");
                conn.rollback();
            } catch (SQLException rollbackEx) {
                rollbackEx.printStackTrace();
            }
        }
    } finally {
        // Close resources
        try {
            if (rs != null) {
                rs.close();
            }
            if (stmt != null) {
                stmt.close();
            }
            if (conn != null) {
                conn.close();
            }
        } catch (SQLException closeEx) {
            closeEx.printStackTrace();
        }
    }
}
}
}

```

Output:

```

List result set for reference....
ID: 1, Age: 20, First: Rita, Last: Tez
ID: 2, Age: 20, First: Sita, Last: Singh
ID: 3, Age: 35, First: Zaid, Last: Khan
ID: 4, Age: 33, First: Sumit, Last: Mittal
ID: 5, Age: 40, First: John, Last: Paul
ID: 7, Age: 20, First: Sita, Last: Singh
ID: 8, Age: 20, First: Rita, Last: Tez
ID: 9, Age: 20, First: Sita, Last: Singh

Deleting row....
Rolled back to savepoint1
Deleting row....
List result set for reference....
ID: 1, Age: 20, First: Rita, Last: Tez
ID: 2, Age: 20, First: Sita, Last: Singh
ID: 3, Age: 35, First: Zaid, Last: Khan
ID: 4, Age: 33, First: Sumit, Last: Mittal
ID: 5, Age: 40, First: John, Last: Paul
ID: 7, Age: 20, First: Sita, Last: Singh
ID: 8, Age: 20, First: Rita, Last: Tez

```

Using SavePoint to rollback to Previous Change Example

In this example, we've six static strings containing a database connection url, username,

password, SELECT Query and two DELETE queries. Delete Queries are used to delete two records in the Employees table with id 8. Now using DriverManager.getConnection() method, we've prepared a database connection. Using setAutoCommit(false), we've set the auto commit as false which is by default true. Once connection is prepared, we've created a Statement object using connection.createStatement() method. While creating Statement Object, we've used ResultSet types as TYPE_SCROLL_INSENSITIVE and CONCUR_READ_ONLY.

Using setSavepoint() method, we've set a save point and then record with id 8 is deleted. As next step, changes are reverted using rollback to earlier savepoint. This will revert the delete row change. Then records are printed using printResultSet() method which prints all the records of a resultset.

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Savepoint;
import java.sql.Statement;

public class SavepointExample2 {
    static final String DB_URL = "jdbc:mysql://localhost/organization";
    static final String USER = "root";
    static final String PASS = "root";
    static final String QUERY = "SELECT id, first, last, age FROM Employees";
    static final String DELETE_QUERY = "DELETE FROM Employees WHERE ID = 8";

    public static void printResultSet(ResultSet rs) throws SQLException {
        // Ensure we start with the first row
        rs.beforeFirst();
        while (rs.next()) {
            // Display values
            System.out.print("ID: " + rs.getInt("id"));
            System.out.print(", Age: " + rs.getInt("age"));
            System.out.print(", First: " + rs.getString("first"));
            System.out.println(", Last: " + rs.getString("last"));
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Connection conn = null;
        Statement stmt = null;
        ResultSet rs = null;

        try {
            // Open a connection
            conn = DriverManager.getConnection(DB_URL, USER, PASS);
            conn.setAutoCommit(false); // Disable auto-commit mode

            stmt = conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
                                         ResultSet.CONCUR_UPDATABLE);

            // Create a savepoint object before executing the delete query
            Savepoint beforeDeleteSavepoint = conn.setSavepoint();
            stmt.executeUpdate(DELETE_QUERY);

            System.out.println("Table EMPLOYEES after executing DELETE_QUERY:");
        }
    }
}
```

```

        rs = stmt.executeQuery(QUERY);
        printResultSet(rs);

        // Rollback the changes after savepoint
        conn.rollback(beforeDeleteSavepoint);

        System.out.println("Table EMPLOYEES after rollback:");
        rs = stmt.executeQuery(QUERY);
        printResultSet(rs);

        // Commit if no issues (optional here)
        conn.commit();

    } catch (SQLException e) {
        e.printStackTrace();
        if (conn != null) {
            try {
                System.out.println("Rolling back transaction due to an
                                   error...");

                conn.rollback();
            } catch (SQLException rollbackEx) {
                rollbackEx.printStackTrace();
            }
        }
    } finally {
        // Clean up resources manually
        try {
            if (rs != null) {
                rs.close();
            }
            if (stmt != null) {
                stmt.close();
            }
            if (conn != null) {
                conn.close();
            }
        } catch (SQLException closeEx) {
            closeEx.printStackTrace();
        }
    }
}

```

Output:

Table EMPLOYEES after executing DELETE_QUERY:

```

ID: 1, Age: 20, First: Rita, Last: Tez
ID: 2, Age: 20, First: Sita, Last: Singh
ID: 3, Age: 35, First: Zaid, Last: Khan
ID: 4, Age: 33, First: Sumit, Last: Mittal
ID: 5, Age: 40, First: John, Last: Paul
ID: 7, Age: 20, First: Sita, Last: Singh

```

Table EMPLOYEES after rollback:

```

ID: 1, Age: 20, First: Rita, Last: Tez
ID: 2, Age: 20, First: Sita, Last: Singh
ID: 3, Age: 35, First: Zaid, Last: Khan
ID: 4, Age: 33, First: Sumit, Last: Mittal
ID: 5, Age: 40, First: John, Last: Paul
ID: 7, Age: 20, First: Sita, Last: Singh
ID: 8, Age: 20, First: Rita, Last: Tez

```